

Social Media Content Analyzer

Recommended Implementation Plan & Project Structure (3-Day Build)

1. Priorities, In Order

- 1 A working core pipeline: upload → extract (PDF/image) → OCR fallback → LLM analysis → structured output.
- 2 A UI that clearly shows loading and error states, and never dies silently on bad input.
- 3 A tight README with a ~200-word approach summary, setup steps, and an honest limitations section.
- 4 Deployment that survives being tested cold by a stranger, on the first try.
- 5 Only after all of the above are solid: additional analysis fields, extra edge cases, extra tests.

2. Tech Stack

Layer	Choice	Why
Backend	FastAPI	Async, fast to build, clean OpenAPI docs for free.
PDF parsing	PyMuPDF (fitz)	Fast, handles mixed text/scanned pages well.
OCR	Tesseract (pytesseract)	Free, no API key, good enough accuracy for typed/scanned text.
LLM	Gemini 2.5 Flash (free tier)	Free tier available, fast, good structured-output support.
Frontend	React + TypeScript + Tailwind (or plain HTML/CSS if the visit is high)	Clear UX with a manageable scope.
Deployment	Single Render service (or Render backend + Vercel frontend parts to break)	Preventing parts to break during evaluation.

3. Project Structure

```
root/
├── app/
│   ├── main.py                # FastAPI app, routes, startup config
│   └── services/
│       ├── extraction.py      # PyMuPDF text extraction + OCR fallback
│       └── analysis.py        # LLM call, prompt, structured parsing
│   └── models/
│       ├── schemas.py         # Pydantic request/response models
│       └── static/ or templates/ # Minimal upload UI if serving frontend together
│   └── config.py              # Env var loading, constants (size limits, etc.)
├── frontend/                  # Only if kept separate (React+TS+Tailwind)
│   ├── components/
│   ├── pages/
│   └── services/
├── tests/
│   └── test_core_paths.py      # 10-12 tests mapped to real edge cases
├── README.md                  # Approach, setup, assumptions, limitations
└── .env.example
```

4. Core Pipeline Logic

extraction.py: try PyMuPDF text extraction first. If a page returns no/near-empty text, render that page to an image and run Tesseract OCR on it. Merge results per document, tagging which pages needed OCR. Cap accepted file size up front; reject unsupported types before doing any parsing work.

analysis.py: split extracted text into individual posts using a simple, documented heuristic (e.g., blank-line separated blocks, or one post per page if that matches the test data). Send each post to the LLM with a prompt that requests a fixed JSON shape. Validate the LLM's JSON against a Pydantic schema before returning it — if validation fails, retry once, then fall back to a clear partial-result error rather than crashing.

5. Analysis Output Fields (Chosen Subset)

- Sentiment (positive / neutral / negative)
- Engagement Score (0-100, with one-line justification)
- Strengths (1-3 bullet points)
- Weaknesses (1-3 bullet points)
- Improved version of the post

Add hashtag suggestions or a readability score only after these five are working reliably end to end.

6. Edge Cases to Implement Directly

- Unsupported file type → reject with a clear message before processing.
- Image-only PDF or scanned image → falls through to OCR automatically.
- No text extracted after OCR → return a clear 'no readable text found' response, not a crash.
- Oversized upload → reject with a stated size limit.
- LLM/API failure or timeout → catch, return a graceful error, retry once if cheap to do.

Remaining cases from the original 14 (duplicates, unicode/emoji edge cases, malformed OCR output, multi-post parsing ambiguity, extremely long text) belong in the README's Limitations section as acknowledged, deliberately deferred items — not silently ignored.

7. Testing Plan (10-12 tests)

- 1 Valid text-based PDF extracts correctly.
- 2 Image-only PDF triggers OCR and returns text.
- 3 Unsupported file type is rejected with the correct error.
- 4 Oversized file is rejected before processing.
- 5 Blank/empty file returns a clear 'no content' response.
- 6 LLM call failure is caught and returns a graceful error, not a 500.
- 7 Structured output always matches the Pydantic schema.
- 8 Multi-post document is split into the expected number of posts.
- 9 Mixed PDF (text + scanned pages) merges both extraction paths correctly.
- 10 End-to-end: upload → response contains all expected analysis fields.

8. README Outline (~1 page)

- Approach summary (~200 words, reusable for the separate write-up)
- Setup & run instructions
- Environment variables (.env.example)
- API description (endpoints, request/response shape)
- Assumptions made (e.g., post-splitting rule)
- Known limitations (deferred edge cases, listed explicitly)
- What I'd do with more time
- Deployment link

9. Suggested 3-Day Timeline

Day	Focus
Day 1 (Jul 11)	Backend skeleton, PDF extraction, OCR fallback working end to end on sample files.
Day 2 (Jul 12)	LLM analysis integration, structured output schema, frontend upload UI with loading/error states.
Day 3 (Jul 13)	Core edge cases, targeted tests, deployment, cold-test the live URL, write README + 200-word summary.
Buffer (Jul 14)	Final cold test from a fresh browser/session before submitting; fix anything that breaks.

10. Differentiators for Competitive Selection (700+ Candidates)

Everything above is the floor: a solid, working, well-scoped submission that likely lands in the top 15-20% by default, since a large share of submissions at this volume will have broken deploys, unhandled crashes, or rushed docs. To move from 'good' to 'selected,' add one or two of the following on top — but only after the core pipeline is rock solid, not instead of it. This section should only be started once Sections 1-9 are done: the pipeline works end to end, core edge cases are handled, tests pass, and the app is deployed and cold-tested.

#	Idea	Effort vs. Payoff
1	Stream the LLM analysis to the frontend (SSE / StreamingResponse) instead of blocking the UI until the full result is ready.	Medium effort, multiple payoffs
2	Show the extraction path taken in the UI (e.g., 'Page 2 required OCR - confidence: 87%') so the edge case high payoff is visible, not just implemented.	Low effort, high payoff
3	Render original vs. improved post side-by-side with changes visually highlighted, instead of a single text block.	Low-mid effort, high payoff
4	Invest real care in prompt engineering and JSON validation: graceful degradation on ambiguous or malformed input, no crashes.	High effort, high payoff
5	One honest, specific trade-off written out in the README (a real edge case caught, the dev's own trade-off, why it's not fixed).	Very low effort, high payoff

Deliberately skip: adding more analysis fields, more edge cases, or more tests purely to stand out. With 700+ candidates, a reviewer's attention per submission is short — volume reads as noise, not craft. The submissions that stand out are the ones where the reviewer clicks the live link, uploads a deliberately weird file, and it just works smoothly, with one or two visible details that show real care.